

Carnet de navigation

Ce projet vise à analyser l'évolution des codes Python produits par des élèves dans des plateformes d'apprentissage de la programmation.

Contexte

Le projet s'inscrit dans l'étude de quatre environnements d'apprentissage de la programmation où les élèves produisent plusieurs versions successives de code pour résoudre un problème.

Objectifs du problème

L'objectif principal est de proposer un système permettant d'identifier et de caractériser les évolutions entre le code initial et le code final soumis par les élèves. Ces modifications doivent être exprimées de manière lisible et pertinente pour un enseignant d'informatique.

Outils proposées

Le projet s'appuie sur plusieurs composantes techniques :

1. **Exploitation de la librairie ast-error-detection** : Cette librairie Python convertit deux codes en AST (Abstract Syntax Trees) et calcule les modifications nécessaires pour passer d'un code à l'autre.
2. **Les ressources fournies** incluent des bases de codes produits par des élèves dans différents contextes. (AlgoPython, Pirate, SPY)

Objectifs

Voici les différents objectifs que nous nous sommes fixés pour répondre aux problèmes

1. **Couche d'interprétation sémantique** : Au lieu de simplement compter les modifications, le système doit pouvoir exprimer les changements en termes significatifs comme l'ajout/suppression de structures de "contrôle" (if, for, while et autres), l'ajout/suppression de variables, ou la modification de valeurs
2. **Analyse des trajectoires** : En prolongement, le projet propose d'identifier des trajectoires de programmation similaires et de classer les groupes de codes pour faire émerger des profils ou stratégies d'évolution
3. **Système de visualisation** : Une interface permettant de visualiser l'évolution des codes de manière compréhensible

Solutions envisagées

Couche d'interprétation sémantique

- Utiliser la librairie ast-error-detection pour générer les AST des différentes versions de code et récupérer les opérations d'édition (insertions, suppressions, substitutions) entre deux soumissions successives.
- Définir une liste de catégories de modifications pédagogiquement parlantes : ajout/suppression de structure de contrôle, ajout/suppression de variable, modification d'expression ou de valeur, restructuration de blocs (indentation, déplacement de code).
- Concevoir des règles qui traduisent les opérations AST en ces catégories (par exemple : insertion d'un nœud "If" → "ajout d'une condition") et implémenter ces règles dans une fonction Python dédiée.
- Intégrer la fonction d'interprétation dans les chose existant (récupération JSON, construction AST, affichage du code)
- Produire une sortie structurée (par exemple un JSON) listant, pour chaque paire de codes, les modifications classées par catégorie, avec éventuellement un niveau de gravité ou d'impact.

Analyse de trajectoires

- Filtrer les itérations de codes d'une même personne selon le problème (code vide, sans changement, ...)
- Comparer une instance à une autre, et mettre en valeur les changements significatifs.
- Pouvoir trouver les différentes catégories d'évolution des codes et les classer dans ces catégories
- Représenter chaque trajectoire d'élève (pour un exercice donné) comme une séquence d'états, chaque état étant décrit par un vecteur de caractéristiques : nombre de modification, succès ou échec, ect...
- Relier des profils à des informations pédagogiques (réussite de l'exercice, nombre total de soumissions, temps passé) pour proposer une interprétation utile aux enseignants.

Système de visualisation

- à déterminer